6 A

1. Concepts to Understand

- **MoSCoW Prioritisation:**
    - This is a method used in Agile projects to rank what work is most important.
    - We cannot do everything at once. Some things are necessary (Vital), and some are just nice to have. MoSCoW helps decide this order.
    - **Flexibility:** This method ensures the most important work gets finished. If the team runs out of time, they drop the less important tasks ("Could Haves") to make sure the "Must Haves" are finished. This creates a "safety buffer" or contingency.
-
- **Timeboxing:**
    - A Timebox is a fixed period of time (like a container of time) where work is done.
    - The time cannot change (it is fixed). If you cannot finish everything, you remove low-priority tasks, but you **never** extend the time.
    - **Goal:** The goal is to deliver a complete and meaningful result at the end of every Timebox (usually every 2-4 weeks).
-
- **Empowered Team:**
    - The team (Solution Development Team - SDT) has the power to make decisions on *how* to do the work inside the Timebox. They do not need to ask the Project Manager for every small decision, as long as they stay within the main goals.
-

---

2. Key Points to Memorize

A. MoSCoW Definitions (Memorize the letters):

- **M - Must Have:** Essential/Critical.
    - Without this, the solution is illegal, unsafe, or does not work.
    - Guaranteed delivery (Non-negotiable).
-
- **S - Should Have:** Important but not vital *now*.
    - Can be postponed or worked around if necessary.
-
- **C - Could Have:** "Nice to have" / Desirable.
    - Less important.
    - These are the first items to be dropped if time is short.
-
- **W - Won't Have this time:** Agreed that this will **not** be delivered in the current timebox.

B. Effort Allocation (Formulas):

- **Must Have:** Maximum **60%** of total effort.
- **Could Have:** Approximately **20%** of total effort (This is your contingency/safety margin).

C. Roles in Prioritisation:

- **Business Visionary & Business Ambassador:** Have the **final say** on what the priorities are.
- **Project Manager (PM) & Business Analyst (BA):** Help guide the discussion but do not make the final business decision.

D. Timeboxing Rules:

- **Duration:** Typically **2 to 4 weeks**.
- **Styles of Timeboxing:**
    - **DSDM Structured Timebox:** Has 3 main steps inside:
        1. Investigation
        2. Refinement
        3. Consolidation
    -
    - **Free Format Timebox:** Used when the structured style is not helpful (similar to a "Scrum Sprint").
-
- **Daily Stand-up:**
    - A short daily meeting (max 15 minutes, approx 2 mins per person).
    - **3 Questions each person answers:**
        1. What have I done since last time?
        2. What will I do until the next meeting?
        3. What problems (blockers) are stopping me?
    -
-

E. Managing Changes:

- **Changes allowed inside the team:** Small changes that do not affect the main objective.
- **Changes requiring formal approval:** Adding new "Must Haves," changing the main goal, or increasing the budget/timeline.

---

3. Final Summary

To pass the exam for this chapter, remember that **Agile projects need strictly prioritized requirements**. We use **MoSCoW** to sort work: **Must Haves** (essential, max 60% effort), **Should Haves** (important but strictly not vital now), **Could Haves** (desirable, 20% effort, acts as safety), and **Won't Haves** (not now).

This work is done inside **Timeboxes**, which are fixed periods of **2-4 weeks**. Timeboxes ensure work is delivered frequently. During a Timebox, the team meets daily (The **Daily Stand-Up**) to answer three specific questions about progress and problems. The Business Visionary decides the priorities, but the Team is responsible for finishing the work on time by adjusting the lower priority items ("Could Haves") if needed.

6 B

1. Concepts to Understand

- **The Case Study (AMS-FR):**
    - The project is to build a system that takes attendance automatically using a camera.
    - It replaces paper sign-in sheets.
    - It identifies students' faces and marks them as "Present," "Late," or "Unverified."
-
- **User Stories (How we write requirements):**
    - In Agile, we don't just write "The system must do X." We write a **User Story** to understand *who* needs it and *why*.
    - **Format:**
        1. **As a** (Role: e.g., Lecturer)
        2. **I want** (Feature: e.g., to record attendance automatically)
        3. **So that** (Benefit: e.g., it is accurate and saves time).
    -
-
- **Functional vs. Non-Functional Requirements:**
    - **Functional (FR):** What the system **does**. (Example: Capturing an image, Saving to a database).
    - **Non-Functional (NFR):** How **well** the system performs. (Example: Speed less than 3 seconds, Accuracy over 90%).
-
- **Applied Timeboxing:**
    - The case study splits work into **Sprints** (Timeboxes).
    - Each Timebox follows the 3 steps from DSDM:
        1. **Investigate:** Planning and analyzing.
        2. **Refine:** Doing the main development work.
        3. **Consolidate:** Testing, fixing bugs, and finishing.
    -
-

---

2. Key Points to Memorize

You must remember how to classify requirements and the steps of a User Story for the exam.

A. User Story Formula

- **Who:** "As a..." (Student, Lecturer, Admin)
- **What:** "I want..." (The feature)
- **Why:** "So that..." (The value or reason)

B. MoSCoW Examples (From the Case Study)
If the exam asks you to categorize features for an attendance system, use these examples:

- **Must Have (Critical):**
    - Capture images from the camera.
    - Match face to student profile.
    - Store data securely (Encryption).
    - Response time under 3 seconds (Performance).
-
- **Should Have (Important but can wait):**

    - Analytics Dashboard (Charts/Graphs).
    - Exporting reports (PDF/CSV).
    - Filters (By semester or program).
-
- **Could Have (Nice to have):**
    - Mobile-friendly view (viewing on a tablet).
    - Auto-generating system health logs.
-
- **Won't Have (Out of scope):**
    - Using fingerprints (Biometrics).
    - Integration with other faculties (Future phase).
-

C. Timebox (Sprint) Schedule
In this case study, a **2-week Sprint** is divided like this:

- **Days 1–3 (Investigate):** Check requirements and design the basics.
- **Days 4–10 (Refine):** Write code, build features, create prototypes.
- **Days 11–14 (Consolidate):** Test, fix bugs (debug), and deploy the finished work.

---

3. Final Summary

This chapter applies the Agile tools to a **Facial Recognition Attendance Project**.

You must understand how to write a **User Story** (*As a... I want... So that...*).
You must know the difference between **Functional** (features like "take photo") and **Non-Functional** (quality like "fast speed").

When applying **MoSCoW**, the core features (like recognizing a face) are **Must Haves**. Analytics and reports are **Should Haves**. Mobile support is a **Could Have**.

Finally, remember the practical **Timebox flow**: Spend the first few days **Investigating**, the middle days **Refining** (building), and the final days **Consolidating** (checking and finishing).

7

1. Concepts to Understand

- **Facilitated Workshops:**
    - Normally, meetings can be messy and waste time.
    - A **Facilitated Workshop** is a special type of meeting designed to get a clear result quickly.
    - It uses a **Neutral Facilitator**: A person who runs the meeting but does **not** give their own opinion on the solution. Their only job is to manage the group so decisions are made without fighting.
-
- **Modelling:**
    - Modelling means **using pictures or diagrams** instead of just words to explain complex ideas.
    - It helps people visualize (see) the problem or solution.
    - Think of it like an architect's blueprint or a sketch. It creates a shared understanding between technical people and business people.
-
- **Iterative Development:**
    - Instead of building the whole product at once (perfectly), you build it in **cycles**.
    - You create a rough version, check it, and then improve it.

The Cycle: Thought (Planning)
→\rightarrow→

Action (Doing)
→\rightarrow→

    -
      Conversation (Reviewing).
    - **Benefits:** It catches mistakes early. If the business changes its mind, the team can adapt easily.
-
- **Reviews and Testing:**
    - **Review (Static):** Reading documents or designs to find errors before building.
    - **Testing (Dynamic):** Actually running the software to see if it works.
-

---

2. Key Points to Memorize

A. Facilitated Workshops

- **Definition:** A structured meeting to reach a specific objective in a short time.
- **Key Success Factors:**
    - Need an **independent Facilitator** (someone neutral).
    - Clear objectives (goals) before starting.
    - Decisions should be agreed upon by the team, not forced.
-
- **Benefits:** Quick decisions, builds team spirit, clarifies issues.

B. Modelling

- **Purpose:** To improve communication and see "the big picture."
- **6 Perspectives (Memorize the Venn Diagram):**
    - **What:** The data/information.

    - **How:** The functions and processes.
    - **Where:** The locations.
    - **Who:** The people (users/stakeholders).
    - **When:** The timing/events.
    - **Why:** The business strategy/objectives.
-

C. Iterative Development Cycle

- **The Cycle inside a Timebox (IRC):**
    1. **Investigate:** Details and Acceptance Criteria.
    2. **Refine:** Develop and code the feature.
    3. **Consolidate:** Final checks and testing to make sure it is ready.
-

D. Roles in Development (Who does what?)

- **Technical Coordinator:** Checks technical quality (Architecture).
- **Solution Tester:** Doing the main testing.
- **Business Ambassador:** Doing the "Business Acceptance Testing" (confirming it is what they wanted).

E. Important "Missing Word" Rule (from the exercises)

- "A DSDM project will always deliver a viable solution, as long as the practices of **MoSCoW** and **Timeboxing** are followed." (This is a common exam question).

---

3. Final Summary

To succeed in this chapter, understand that DSDM uses **Facilitated Workshops** to make decisions fast using a neutral leader. It uses **Modelling** (pictures and diagrams) to explain the **Who, What, Where, When, Why, and How** of a project.

The building process is called **Iterative Development**. We do not build perfectly the first time; we build in cycles (**Investigate**

```
→\rightarrow→
```

Refine
→\rightarrow→

Consolidate) inside short Timeboxes. Quality is never compromised; it is checked constantly through **Reviews** (reading code/docs) and **Testing** (running the system).

8

1. Concepts to Understand

- **What is Software Estimation?**
  - It is the process of predicting three things:
    1. **Effort:** How much human work is needed (e.g., person-hours).
    2. **Time:** How long the project will take (duration/schedule).
    3. **Cost:** How much money is required.
  -
  - **Goal:** To reduce uncertainty so teams can plan budgets and timelines without guessing.
-
- **Difference between Effort, Duration, and Cost:**
  - **Effort:** The actual work. (Example: "This task takes 8 hours of work").
  - **Duration:** The calendar time. (Example: "This task takes 2 days because we have meetings in between").
  - **Cost:** The money spent on salaries and tools.
-
- **Top-Down vs. Bottom-Up Estimation:**
  - **Top-Down:** Looking at the "Big Picture." You estimate the whole project quickly using comparisons. (Used early in the project).
  - **Bottom-Up:** Looking at the details. You break the project into tiny tasks, estimate each one, and add them up. (Used later when you know more details).
-
- **Key Techniques Explained:**
  - **T-Shirt Sizing:** You don't use numbers. You group tasks by size: Small (S), Medium (M), Large (L), or Extra Large (XL). It is very fast.
  - **Velocity:** In Agile, you measure how fast your team worked in the past (e.g., "We finished 20 points last sprint"). You assume the team will move at the same speed in the future.
  - **Function Points:** You measure the size of software by counting functions (Inputs, Outputs, Files) rather than lines of code. It focuses on what the user sees.
-

---

2. Key Points to Memorize

A. Factors Affecting Costs (5 Main Areas):

1. **Product:** Size (SLOC/Story Points), Complexity, Quality needs.
2. **Team (Human): Experience is the #1 factor**. (Also includes skill level and communication overhead).
3. **Technical:** Complexity of algorithms, multiple systems working together.
4. **Time/Scheduling:** Short deadlines add pressure and cost.
5. **Organizational Maturity:** Processes like CMMI make costs more predictable.

B. Types of Estimation Techniques (Categorization):

- **Top-Down (Macro):**
  1. T-Shirt Sizing.
  2. Function Point Estimating.
  3. Velocity-Based Estimation.
-
- **Bottom-Up (Micro):**

1. Product Breakdown Estimating.
2. Work Breakdown Estimating.

-

C. Definitions & Rules:

- **Velocity Formula:** Average number of **Story Points** a team delivers per Timebox/Sprint.
    1. *Note: Velocity is unique to one team; you cannot compare Team A's velocity to Team B's.*
-
- **5 Components of Function Points (FP):**
    1. **External Inputs (EI)**
    2. **External Outputs (EO)**
    3. **External Inquiries (EQ)**
    4. **Internal Logical Files (ILF)**
    5. **External Interface Files (EIF)**
-
- **Brooks's Law Effect:** Adding more people to a late project can make it later because of **communication overhead**.

D. Calculation Logic (From Examples):

- **T-Shirt to Days:** If "S = 1 day" and "M = 3 days", then total effort is simple multiplication.
- **Velocity Forecasting:**
    - *Formula:* Total Points Required ÷ Average Velocity = Number of Sprints needed.
-
- **Bottom-Up Sum:** Task A + Task B + Task C = Total Effort.

---

3. Final Summary

Software Estimation predicts **Effort, Time, and Cost**. It is not just guessing; it uses structured methods.

For the exam, know the difference between **Top-Down** methods (Fast, high-level) like **T-Shirt Sizing** and **Velocity**, and **Bottom-Up** methods (Detailed, slower) like **Work Breakdown**.

Remember that **Human Experience** is the biggest factor in cost accuracy. If using **Function Points**, you are measuring functionality (Inputs/Outputs), not code. If using **Velocity**, you are using past performance to predict the future. All methods aim to stop projects from running out of money or time.

FINAL preapared

📚 THE AGILE PROJECT MANAGEMENT FINAL EXAM MASTER GUIDE

Subject Overview:
This subject is about **DSDM (Dynamic Systems Development Method)**, which is a specific way of doing **Agile Project Management**.

- **Old way (Traditional):** You plan everything for months, then build for months, and hope it works.
- **New way (Agile/DSDM):** You build small pieces of the software quickly (in 2-4 weeks), check if they work, and then build the next piece. This allows you to change your mind if needed.

This guide covers **4 Modules**:

1. **Prioritising & Scheduling** (MoSCoW & Timeboxing)
2. **Real Application** (Case Study: Facial Recognition)
3. **DSDM Practices** (Workshops, Modelling, Iterative Development)
4. **Estimation** (Calculating Cost and Time)

MODULE 1: DSDM BASICS (MoSCoW & Timeboxing)

To manage a project where things might change, you need two things: a way to decide **what is important** (MoSCoW) and a strict **schedule** (Timeboxing).

Part A: MoSCoW Prioritisation

We cannot do everything. If we try to do everything, the project will be late. We must rank our requirements. DSDM uses a technique called **MoSCoW**.

1. What do the letters mean? (MEMORIZE THIS)

- **M = Must Have (The Criticals)**
  - **Definition:** These are requirements that contain the **Minimum Usable Subset**.
  - **Rule:** If you do not deliver a "Must Have," the project is a **failure**.
  - **Criteria:** Ask yourself:
    - Is it illegal without this?
    - is it unsafe without this?
    - Is the solution useless without this?
    - **Key Question:** "If we don't have this on the final night, will you cancel the project?" If the answer is YES, it is a Must Have.

    - **Target Effort:** "Must Haves" should not take more than **60%** of your total effort. This ensures you have safety time for other things.
-
- **S = Should Have (The Important but Not Critical)**
    - **Definition:** Important but not vital *right now*.
    - **Rule:** It would be painful to leave these out, but the solution will still work.
    - **Workaround:** You can usually find a manual workaround (e.g., using a spreadsheet instead of an automated report) temporarily.
-
- **C = Could Have (The Nice-to-Haves)**
    - **Definition:** Wanted or desirable, but less important.
    - **Rule:** These are your **Safety Buffer (Contingency)**.
    - **Role in Planning:** Ideally, "Could Haves" act as **20%** of your total effort. If the project runs out of time, you simply **drop** the "Could Haves" to save the "Must Haves."
-
- **W = Won't Have this time**
    - **Definition:** These are requirements the team agrees will **not** be delivered in this timeframe (or Sprint).
    - **Strategy:** They might be done in the future, but they are out of scope for now.
-

2. How to Prioritise Successfully

- **Who Decides?** The **Business Visionary** and **Business Ambassador** (the business people) have the final say. The Project Manager (PM) facilitates, but does not decide.
- **Negotiating:** A "Must Have" is non-negotiable.
- **When New Work Appears:** If a new requirement comes in, you must assess its impact. If you add a new "Must," you probably have to remove a "Could" to balance the effort.

Part B: Timeboxing

A **Timebox** is a fixed period of time used to complete a specific amount of work. In other methods (like Scrum), this is often called a **Sprint**.

1. The Golden Rules of Timeboxing

- **Duration:** Usually **2 to 4 weeks**.
- **Fixed Time:** You **never** extend the time. The deadline is fixed.
    - *If you are running late:* You remove tasks (features).
    - *You do NOT:* Add more time.
-
- **Empowerment:** The Solution Development Team (SDT) has the power to decide *how* to do the work inside the Timebox without asking the boss for every detail.

2. Structure of a DSDM Timebox (MEMORIZE THE STEPS)

A Timebox is not just "working for 2 weeks." It has 3 specific stages:

1. **Kick-off:** The team agrees on what they will do.
2. **Investigation (10-20% of effort):** Planning, checking details, and looking at dependencies.
3. **Refinement (60-80% of effort):** This is the main work. Coding, developing, and building the solution.
4. **Consolidation (10-20% of effort):** Testing, fixing bugs, and ensuring everything is "done."
5. **Close-out:** Formal sign-off. Did we succeed? What did we learn?

(Note: There is also a "Free Format" timebox which acts more like a generic Scrum Sprint, but the Structured one above is more unique to DSDM).

3. The Daily Stand-Up

This is a short meeting held every day during a Timebox.

- **Attendees:** The whole Solution Development Team (SDT). The Project Manager observes.
- **Duration:** Max **15 minutes** (2 minutes per person).
- **The 3 Questions (MEMORIZE THESE):**
    1. What have I done since the last meeting?
    2. What will I do until the next meeting?
    3. What **problems/blockers** are stopping me?
-

4. Handling Change in a Timebox

- **The Team's Power:** If a change is small and stays within the "objective" of the timebox, the team can just do it. They don't need formal permission.
- **Formal Change Control:** Needed only if:
    - Someone wants to change the main Goal.
    - Someone wants to add a major "Must Have."
    - The "Must Haves" cannot be finished.
-

MODULE 2: APPLIED CASE STUDY (User Stories)

This module uses a specific example: **Attendance Management System using Facial Recognition (AMS-FR)**. This is a system where cameras take attendance in a classroom automatically.

You need to understand how we write requirements for this system.

Part A: User Stories

In Agile, we don't just write a list of rules. We write "Stories" to explain *who* needs it and *why*.

The Formula (MEMORIZE THIS STRUCTURE):

1. **As a** <Role> (Who is it for?)
2. **I want** <Feature> (What do they want?)
3. **So that** <Benefit> (Why do they want it?)

Example from Case Study:

- **As a** Lecturer...
- **I want** the system to automatically record student attendance using facial recognition...
- **So that** attendance is captured accurately without manual effort.

Part B: Types of Requirements (FR vs. NFR)

1. Functional Requirements (FR)

This describes **what the system does**. It is a visible function.

- *Case Study Examples:*
    - Capture image from camera (FR).
    - Match face to database (FR).
    - Allow lecturer to update attendance (FR).
    - Generate a report (FR).
-

2. Non-Functional Requirements (NFR)

This describes **how well the system works** (Quality, Speed, Security).

- *Case Study Examples:*
    - **Performance:** Recognition must happen in **< 3 seconds**.
    - **Accuracy:** Must be correct **90%** of the time.
    - **Security:** Data must be encrypted.
    - **Availability:** System must be online **99%** of the time during lectures.

Part C: Acceptance Criteria

This tells the developer exactly when the work is finished ("Done").

- *Example:* "Given a student walks into class, When the camera detects a face, Then the system must match it to a profile and mark them 'Present'."

MODULE 3: DSDM PRACTICES

Beyond MoSCoW and Timeboxing, DSDM uses specific techniques to make work faster and clearer.

Practice 1: Facilitated Workshops

Standard meetings are often boring and result in no decisions. A **Facilitated Workshop** fixes this.

- **Definition:** A structured group meeting designed to reach a clear goal in a short time.
- **The Facilitator:** This is the most important person.
    - They are **Neutral** (like a Referee).
    - They manage the people and the time.
    - They do **NOT** give their opinion on the technical solution. Their job is process, not content.
-
- **Benefits:** Faster decisions, high-quality results, and it builds "team spirit" because everyone agrees together.

Practice 2: Modelling

Modelling means "drawing pictures" (diagrams) instead of just writing long text documents.

- **Why use Modelling?**
    1. "A picture is worth a thousand words."
    2. It simplifies complex systems.
    3. It helps non-technical people understand technical ideas.
-
- **Modelling Perspectives (MEMORIZE THE LIST):**
  We model the system from 6 angles:
    1. **WHAT:** The data and information.
    2. **HOW:** The functions and processes.
    3. **WHERE:** The locations.
    4. **WHO:** The people/users.
    5. **WHEN:** The timing and events.
    6. **WHY:** The business objectives.

Practice 3: Iterative Development

"Iterative" means repeating a cycle to improve something.
We do not build the whole software perfectly in one go. We build a rough version, check it, and then refine it.

- **The Cycle (IRC):** Inside every timebox, we follow these steps:
    - **Investigate:** What are we building? (Details & planning).
    - **Refine:** Build it! (Coding and development).
    - **Consolidate:** Check it! (Testing and quality assurance).
-
- **The Motto:** "Thought, Action, Conversation."
- **Roles:**
    - **Solution Tester:** Does the technical testing.
    - **Technical Coordinator:** Checks technical quality and architecture.
-
- **Core Rule:** "Never compromise quality." Even though we work fast, the software must work well.

MODULE 4: SOFTWARE ESTIMATION (Calculations)

Software Estimation is predicting how much **Effort, Time, and Cost** a project needs.

Part A: Concepts to Understand

- **The Triple Challenge:** Software estimation is hard because requirements change, complexity is hidden, and humans are bad at guessing.
- **Effort vs. Duration vs. Cost:**
    - **Effort:** How many hours of work (e.g., "10 man-hours").
    - **Duration:** How long it takes on a calendar (e.g., "3 days," because we don't work 24 hours a day).
    - **Cost:** How much money is spent (Salaries + Resources).
    - *Important:* Adding more people to a late project often makes it later (This is called **Brooks's Law**) because of communication overhead.
-

Part B: Factors Affecting Cost (5 Groups)

1. **Product:** How big is it? (Size). How complicated is the logic? (Complexity).
2. **People:** How experienced is the team? (**Team Experience** is the #1 factor in accurate estimation).
3. **Technology:** Are we integrating with old legacy systems? That is harder.
4. **Time:** Is the deadline too tight? That increases cost (stress/mistakes).
5. **Organization:** Mature organizations (those with clear processes) estimate better.

Part C: Estimation Techniques (Top-Down)

"Top-Down" means looking at the big picture without knowing every detail yet. It is fast but rough.

1. T-Shirt Sizing

- **How it works:** Instead of hours, you give items a size: **S, M, L, XL**.
- **Usage:** Used very early in the project (planning phase).
- **Calculation Example:**
    - Team agrees: Size S = 1 day effort. Size M = 3 days. Size L = 5 days.
    - If you have 2 Small stories and 1 Large story:
    - (2 * 1) + (1 * 5) = 7 days total effort.
-

2. Velocity-Based Estimation

- **What is Velocity?** It is the measure of how much work a team can complete in one Sprint/Timebox.
- **Unit:** Usually measured in **Story Points** (points representing difficulty).
- **Forecasting Formula:**
    - *Question:* We have 100 points of work remaining. Our velocity is 20 points per Sprint. How many Sprints do we need?
    - *Calculation:* 100 / 20 = **5 Sprints**.
-
- *Note:* You cannot compare Velocity between Team A and Team B. It is unique to the team.

3. Function Point (FP) Estimation

- **Concept:** Measures the "size" of the software by counting features, not lines of code. It looks at functionality from the user's point of view.
- **The 5 Components (MEMORIZE):**
    1. **External Inputs (EI):** User sends data in.
    2. **External Outputs (EO):** System sends reports/data out.
    3. **External Inquiries (EQ):** User asks a question (search).
    4. **Internal Logical Files (ILF):** Data stored inside the system.
    5. **External Interface Files (EIF):** Data referenced from another system.
-

Part D: Estimation Techniques (Bottom-Up)

"Bottom-Up" means breaking the work into tiny details and adding them up. It is slow but accurate.

1. Product Breakdown

- You take the system (The Product) and break it into pieces (Components).
- *Example:* Attendance System -> Interface Module + Database Module + Analytics Module. Estimate each module separately.

2. Work Breakdown (WBS)

- You break the work into **Tasks**.
- *Example:* "Analytics Module" becomes -> "Design chart," "Code query," "Test button."
- This is very detailed and accurate but takes a long time.

🛑 FINAL EXAM "MUST MEMORIZE" CHEATSHEET

Read this section 5 times before entering the exam.

1. ACRONYMS & DEFINITIONS

- **MoSCoW:** Must Have, Should Have, Could Have, Won't Have (this time).
- **DSDM:** Dynamic Systems Development Method.
- **SDT:** Solution Development Team (empowered to make decisions).
- **FR:** Functional Requirement (What it does).
- **NFR:** Non-Functional Requirement (How well it works - speed/security).
- **Velocity:** Average story points delivered per Timebox.

2. THE RULES OF MOSCOW

- **Must Have:** Critical. Max **60%** of total effort. Without it, project fails.
- **Should Have:** Important but can wait. Can be worked around.
- **Could Have:** Nice to have. Approx **20%** of total effort. **Dropped** if we run out of time (Contingency).
- **Won't Have:** Not this time.

3. THE TIMEBOX STEPS

1. **Kick-off**
2. **Investigation**
3. **Refinement**
4. **Consolidation**
5. **Close-out**

- *Remember: Duration is fixed. Quality is non-negotiable. Scope is flexible.*

4. THE 3 DAILY STAND-UP QUESTIONS

1. What did I do yesterday?
2. What will I do today?
3. What are my blockers/problems?

5. MODELLING PERSPECTIVES (Venn Diagram)

- What, How, Where, Who, When, Why.

6. ITERATIVE DEVELOPMENT CYCLE

- **Cycle:** Thought -> Action -> Conversation.
- **Steps:** Investigate -> Refine -> Consolidate (IRC).

7. COST ESTIMATION

- **Top-Down Methods:** T-Shirt, Velocity, Function Points.
- **Bottom-Up Methods:** Work Breakdown Structure (WBS), Product Breakdown.
- **Brooks's Law:** Adding people to a late project makes it later.
- **Function Point Components:** Inputs, Outputs, Inquiries, Internal Files, Interface Files.

📝 EXAMPLE EXAM QUESTIONS & ANSWERS (Simulated)

Q1: What happens if the team runs out of time in a DSDM Timebox?

- **Answer:** They **drop the lowest priority features (Could Haves)**. They do NOT extend the deadline. The deadline is fixed.

Q2: Which role is responsible for ensuring the technical quality of the solution during Iterative Development?

- **Answer:** The **Technical Coordinator**.

Q3: Describe a "Should Have" requirement.

- **Answer:** It is important but not vital for the current timeframe. Leaving it out may be painful, but there is usually a manual workaround.

Q4: A Facilitated Workshop requires a "Neutral Facilitator." What does "Neutral" mean?

- **Answer:** It means the Facilitator manages the process (time, people, arguments) but does not give their own opinion on the technical content or business decision.

Q5: In Function Point Analysis, what is an "External Output"?

- **Answer:** Data or reports that go *out* of the system to the user (e.g., printing an attendance report).

Q6: Calculate the Velocity. Sprint 1 = 18 points. Sprint 2 = 22 points. Sprint 3 = 20 points. What is the velocity?

- **Answer:** (18 + 22 + 20) ÷ 3 = **20 points per Sprint**.

Q7: "As a student, I want to see my attendance so that I know if I am failing." What is this called?

- **Answer:** A **User Story**.

Q8: "The system must load in under 2 seconds." Is this FR or NFR?

- **Answer: NFR (Non-Functional Requirement)**, because it describes performance (speed), not a function.

HOW TO STUDY THIS PAPER

1. **Read Module 1 first.** Understand the MoSCoW rules thoroughly.
2. **Read Module 2.** Visualize the "Classroom Attendance" system to help the theory make sense.
3. **Read Modules 3 & 4.**
4. **Spend the last hour** focusing **only** on the **"Final Exam Cheatsheet"** section. Memorize the lists (The 5 Timebox steps, the 3 Stand-up questions, the letters M-S-C-W).

Good luck. You have everything you need in this document to pass. Stay calm and remember: **Fix the Time, Flex the Scope (Features).**